Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
| Wydział Inżynierii Mechanicznej i Robotyki | Katedra Robotyki i Mechatroniki

INŻYNIERIA MECHATRONICZNA | I stopień | semestr VI | 2025/2026

Programowanie Obiektowe i Inżynieria Oprogramowania

Instrukcja C++/CLI:
2. Praca z kontrolkami, dynamiczne tworzenie obiektów – dodawanie kubeczków

Wprowadzenie do aplikacji okienkowych w C++

Nauczysz się:
Pisać kod pod aplikacje okienkowe w języku C++/CLI z wykorzystaniem platformy *.NET framework* w środowisku *Visual Studio*:

- dodasz obsługę opcji dostępnych w menu,
- dodasz dynamicznie utworzone obiekty – polecenie *gcnew*,
- obsłużysz kliknięcie myszy w wybrane elementy GUI – zmienisz stan programu,
- na podstawie stanu programu określisz wygląd i zachowanie elementów GUI.

Dodatkowe materiały:

- materiały do zajęć: [link],
- oficjalna strona C++/CLI: [link].

Kierunkowe efekty kształcenia pokryte w tej instrukcji:
IME1A_U1, IME1A_U4

Koordynator przedmiotu:
dr inż. Lucjan Miękina, miekina@agh.edu.pl

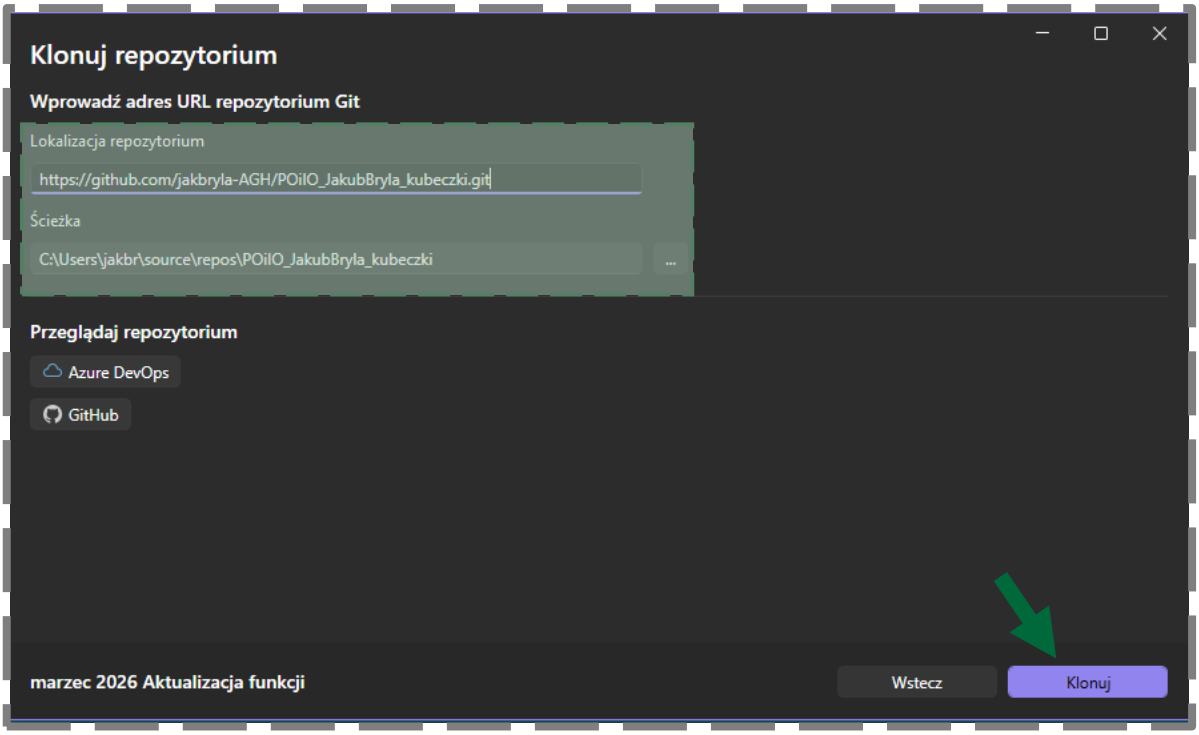
Autor instrukcji:
dr inż. Jakub Bryła, jakub.bryla@agh.edu.pl

1. Wprowadzenie

1. Utworzenie projektu w Visual Studio poprzez repozytorium zdalne

Kontynuując pracę na nowym komputerze lub wirtualnej maszynie należy pobrać aktualny stan projektu z repozytorium zdalnego. W przypadku, gdy projekt nie znajduje się na docelowym komputerze należy otworzyć Virtual Studio i w oknie powitalnym wybrać opcję „Klonuj repozytorium”:

W następnym oknie należy wstawić adres URL repozytorium zdalnego i wybrać lokalizację repozytorium lokalnego. Finalnie, kliknij przycisk „Klonuj”:

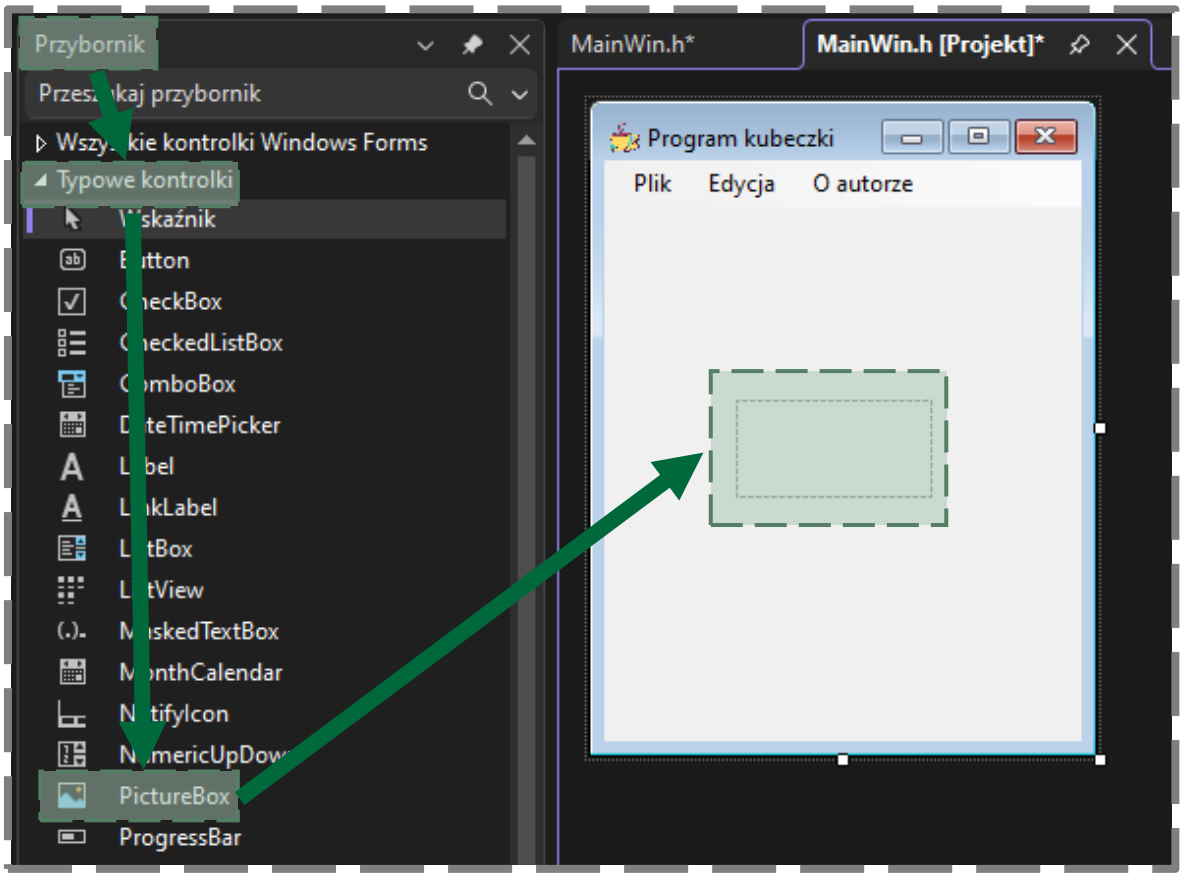


Powinieneś mieć możliwość skompilowania programu i uruchomieniu aplikacji w wersji z poprzednich zajęć.

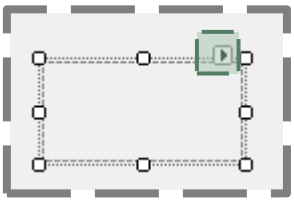
2. Zapoznanie się z kontrolkami GUI i ich użyciem w programie

Przed rozpoczęciem pisania właściwego kodu sprawdzimy kilka zależności między tworzeniem GUI metodą *drag&drop* oraz automatycznie generowanym kodem, sprawdzimy jak pewne funkcjonalności realizowane są w kodzie języka C++/CLI.

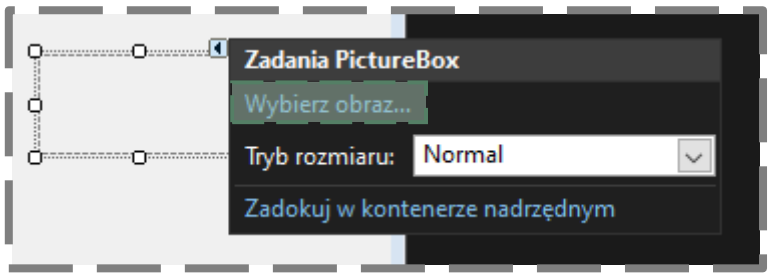
Na początku dodajmy do projektu elementy *PictureBox*:



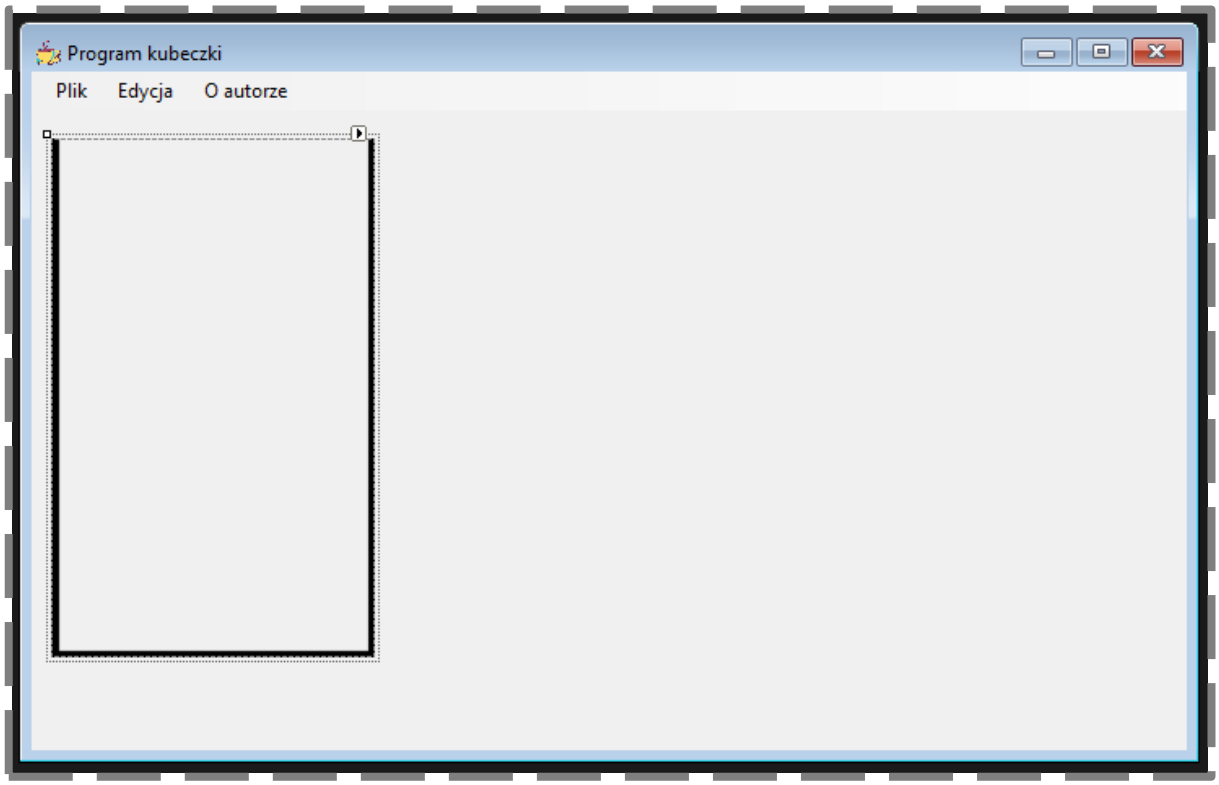
Następnie klikamy raz lewym przyciskiem myszy (LPM) na utworzony ***PictureBox***. U góry elementu pojawi się strzałeczka:



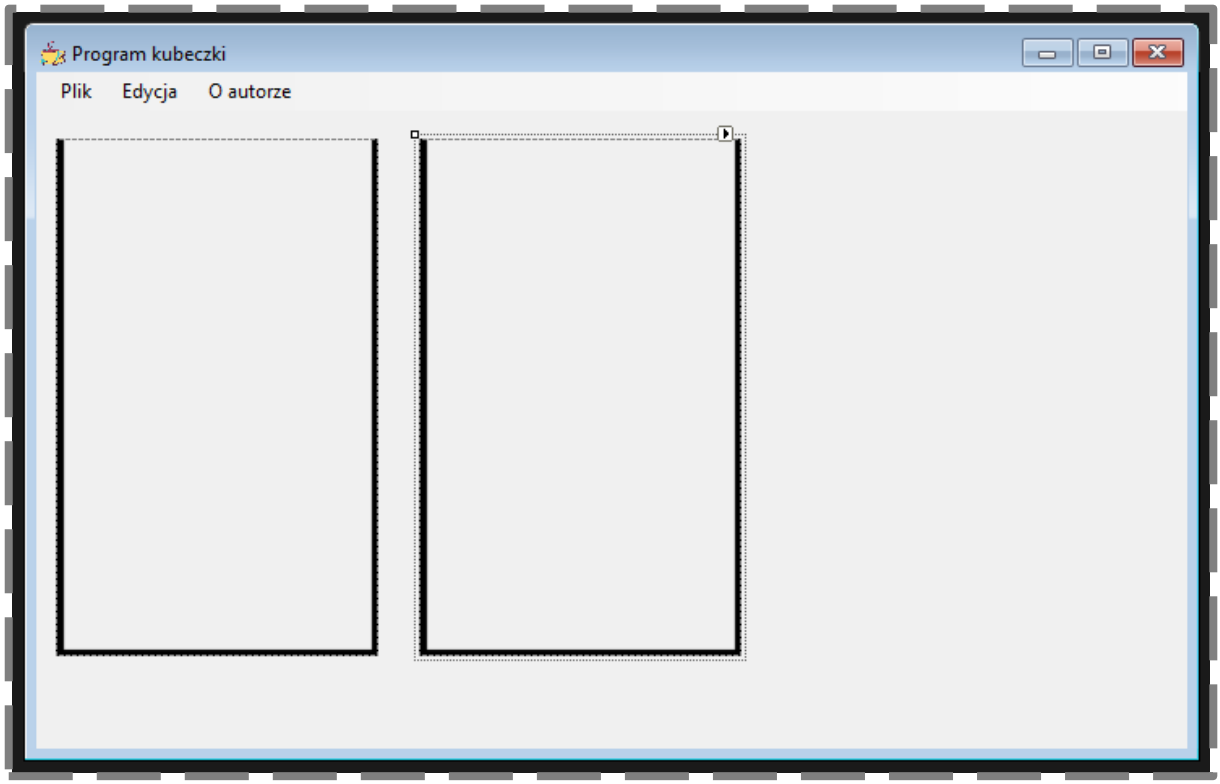
Kliknij raz LMP na strzałkę i wybierz opcję „Wybierz obraz”:



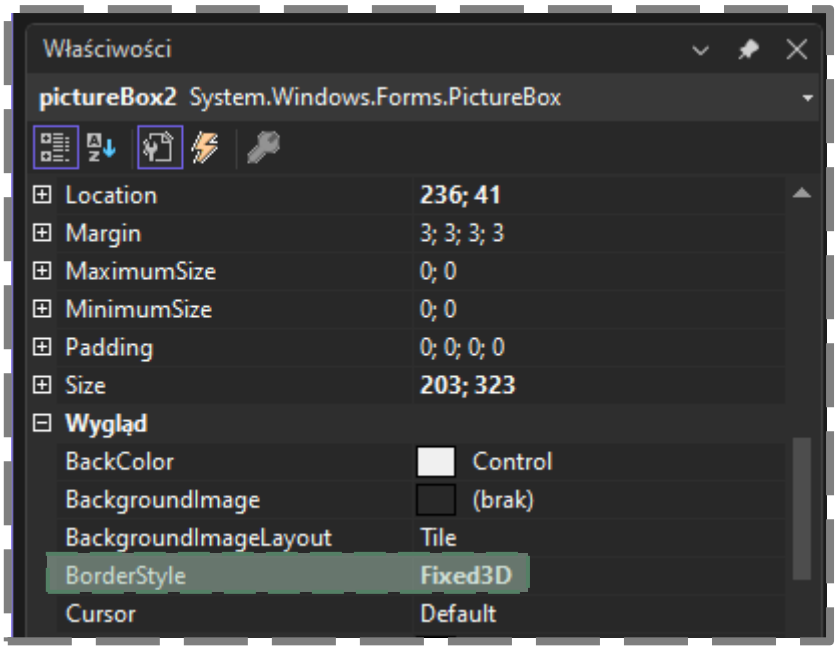
Skopiuj obrazek „cup.png”, dołączony do repozytorium instrukcji, do folderu „*assets*”. Wybierz wspomniany obrazek z lokalizacji projektu. Na powyższym zrzucie ekranu widać opcję „Tryb rozmiaru” – zmień opcję na *AutoSize*. Powiększ okno aplikacji, aby zapewnić miejsce na wyświetlenie kilka kubków:



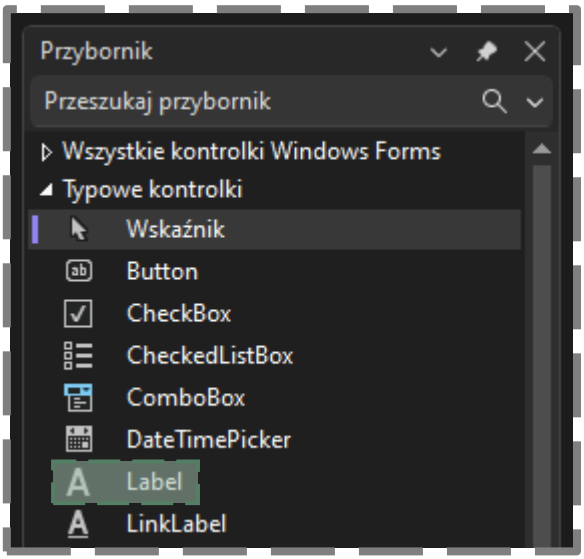
Za pomocą skrótów „***Ctrl + C***” i „***Ctrl + V***” skopiuj kubek i umieść je obok siebie:



Dla drugiego kubka, w zakładce ***Właściwości***, zmień opcję *BorderStyle* na „*Fixed3D*”:

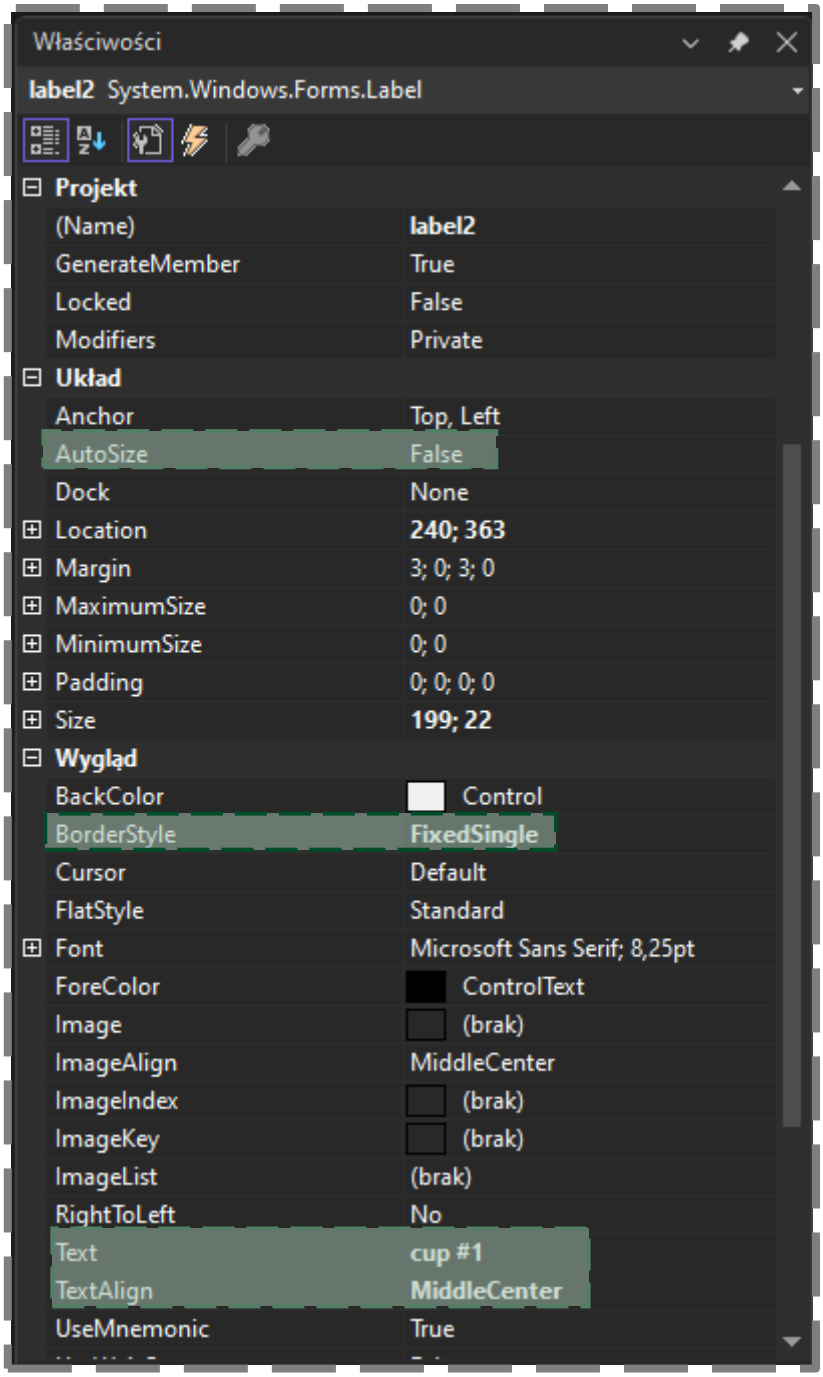


Utwórzmy teraz podpisy pod każdym z kubków. W tym celu dodaj kontrolkę ***Label***:

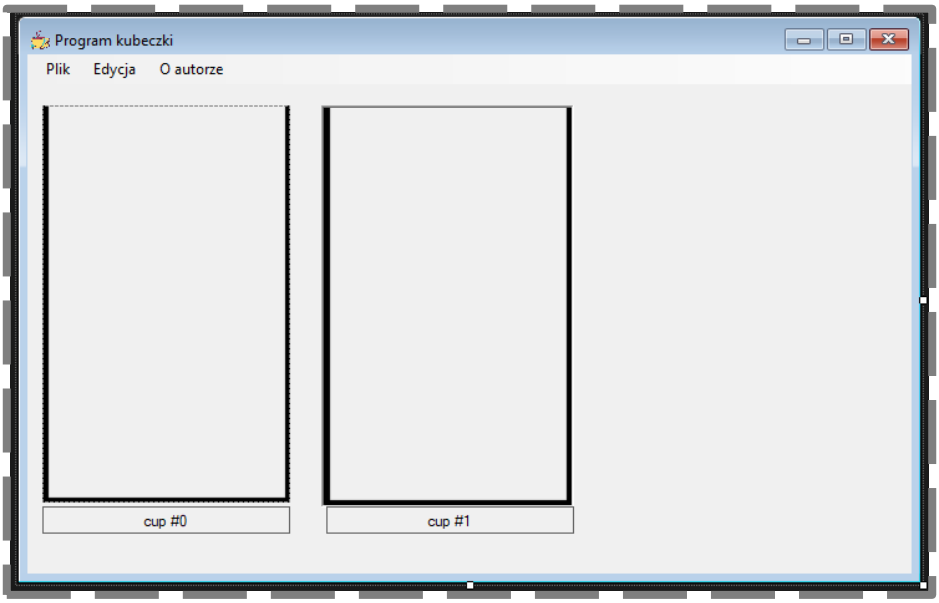


Zmień opcję kontrolki w zakładce ***Właściwości***:

- *AutoSize* na *false*,
- *BorderStyle* na *„FixedSingle”,*
- *Text* na „*cup #0*”,
- *TextAlign* na „*MiddleCenter*”.

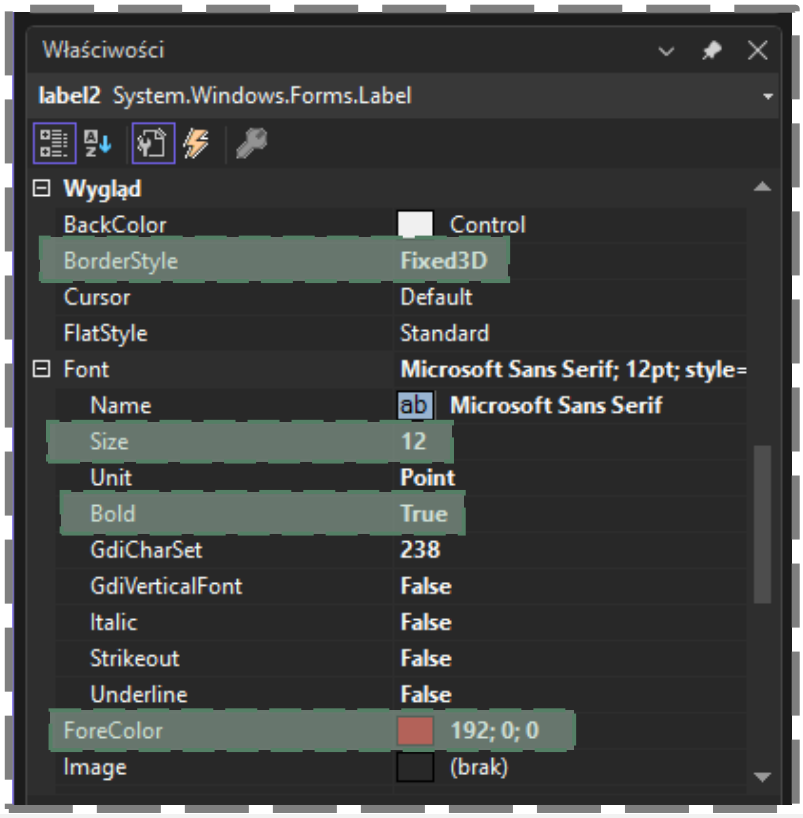


Zmień pozycję kontrolki w oknie aplikacji – zwiększ jej szerokość do szerokości obrazka *cup.png* i wyrównaj elementy względem siebie. Skopiuj podpis kubka, zmień jego nazwę i umieść pod drugim rysunkiem. Efekt powyższych działań:

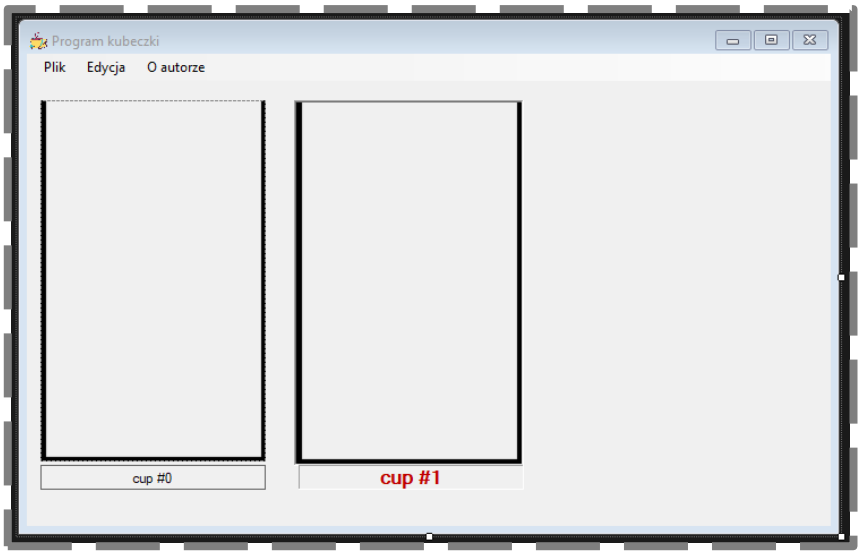


Dla drugiego podpisu zmieńmy ustawienia w zakładce ***Właściwości***:

- *BorderStyle* na „*Fixed3D*”,
- *Font-Size* na „*12*”,
- *Font-Bold* na „*True*”,
- *ForeColor* na „*192;0;0*” – wybierz dowolny kolor, inny niż czarny, z palety kolorów.



Aktualny widok aplikacji:



Uzyskaliśmy dwa interesujące nas widoki kubków i ich podpisów:

1. kubka nieaktywnego „*cup #0*”,
2. kubka aktywnego „*cup #1*” – jest to kubek wybrany przez jednokrotne kliknięcie LPM na kontrolkę ***Label*** kubka.

Zobaczmy istotne elementy kodu wygenerowane automatycznie podczas tworzenia i edycji elementów GUI:

MainWin.h* | MainWin.h [Projekt]*

POiIO kubeczki | POiIOkubeczki::MainWin

```
1   #include <vector>
2   #include "OAutorze.h"
3
4   #pragma once
5
6   namespace POiIOkubeczki {
7
8       using namespace ...
14
15      /// ...
18      public ref class MainWin : public System::Windows::Forms::Form
19      {
20      public:
21          MainWin(void) { ... }
28
29      protected:
30          /// ...
33          ~MainWin() { ... }
40      private: System::Windows::Forms::MenuStrip^ menuStrip1;
41      private: System::Windows::Forms::ToolStripMenuItem^ plikToolStripMenuItem;
42      private: System::Windows::Forms::ToolStripMenuItem^ zamknijToolStripMenuItem;
43      private: System::Windows::Forms::ToolStripMenuItem^ edycjaToolStripMenuItem;
44      private: System::Windows::Forms::ToolStripMenuItem^ substancjeToolStripMenuItem;
45      private: System::Windows::Forms::ToolStripMenuItem^ dodajDoMenuToolStripMenuItem;
46      private: System::Windows::Forms::ToolStripMenuItem^ statystykaToolStripMenuItem;
47      private: System::Windows::Forms::ToolStripMenuItem^ kubekToolStripMenuItem;
48      private: System::Windows::Forms::ToolStripMenuItem^ dodajKubekToolStripMenuItem;
49      private: System::Windows::Forms::ToolStripMenuItem^ usuńKubekToolStripMenuItem;
50      private: System::Windows::Forms::ToolStripMenuItem^ dodajSubstancjeToolStripMenuItem;
51      private: System::Windows::Forms::ToolStripMenuItem^ oAutorzeToolStripMenuItem;
52      private: System::Windows::Forms::PictureBox^ pictureBox1;
53      private: System::Windows::Forms::PictureBox^ pictureBox2;
54      private: System::Windows::Forms::Label^ label1;
55      private: System::Windows::Forms::Label^ label2;
```

Na powyższym zrzucie ekranu widzimy utworzenie zmiennych do przechowywania kontrolek ***PictureBox*** oraz ***Label***.

```
MainWin.h*   MainWin.h [Projekt]*
POiIO kubeczki                    POiIOkubeczki::MainWin
71      #pragma region Windows Form Designer generated code
72          /// ...
76          void InitializeComponent(void)
77          {
78              System::ComponentModel::ComponentResourceManager^ resources = (gcnew System::ComponentModel::ComponentRes
79              this->menuStrip1 = (gcnew System::Windows::Forms::MenuStrip());
80              this->plikToolStripMenuItem = (gcnew System::Windows::Forms::ToolStripMenuItem());
81              this->zamknijToolStripMenuItem = (gcnew System::Windows::Forms::ToolStripMenuItem());
82              this->edycjaToolStripMenuItem = (gcnew System::Windows::Forms::ToolStripMenuItem());
83              this->substancjeToolStripMenuItem = (gcnew System::Windows::Forms::ToolStripMenuItem());
84              this->dodajDoMenuToolStripMenuItem = (gcnew System::Windows::Forms::ToolStripMenuItem());
85              this->statystykaToolStripMenuItem = (gcnew System::Windows::Forms::ToolStripMenuItem());
86              this->kubekToolStripMenuItem = (gcnew System::Windows::Forms::ToolStripMenuItem());
87              this->dodajKubekToolStripMenuItem = (gcnew System::Windows::Forms::ToolStripMenuItem());
88              this->usuńKubekToolStripMenuItem = (gcnew System::Windows::Forms::ToolStripMenuItem());
89              this->dodajSubstancjeToolStripMenuItem = (gcnew System::Windows::Forms::ToolStripMenuItem());
90              this->oAutorzeToolStripMenuItem = (gcnew System::Windows::Forms::ToolStripMenuItem());
91              this->pictureBox1 = (gcnew System::Windows::Forms::PictureBox());
92              this->pictureBox2 = (gcnew System::Windows::Forms::PictureBox());
93              this->label1 = (gcnew System::Windows::Forms::Label());
94              this->label2 = (gcnew System::Windows::Forms::Label());
95              this->menuStrip1->SuspendLayout();
96              (cli::safe_cast<System::ComponentModel::ISupportInitialize^>(this->pictureBox1))->BeginInit();
97              (cli::safe_cast<System::ComponentModel::ISupportInitialize^>(this->pictureBox2))->BeginInit();
98              this->SuspendLayout();
```

Na powyższym zrzucie ekranu został zamieszczony kod odpowiedzialny za utworzenie kontrolek i przypisanie ich do wcześniej zdefiniowanych zmiennych.

Elementy GUI w języku C++/CLI tworzone są przez wywołanie operacji:

```
gcnew System::Windows::Forms::<kontrolka>
```

Ustawienia dla nieaktywnego kubka:

```
192             //
193             // pictureBox1
194             //
195             this->pictureBox1->Image = (cli::safe_cast<System::Drawing::Image^>(resources->GetObject(L"pictureBox1.Image")));
196             this->pictureBox1->Location = System::Drawing::Point(12, 41);
197             this->pictureBox1->Name = L"pictureBox1";
198             this->pictureBox1->Size = System::Drawing::Size(199, 319);
199             this->pictureBox1->SizeMode = System::Windows::Forms::PictureBoxSizeMode::AutoSize;
200             this->pictureBox1->TabIndex = 1;
201             this->pictureBox1->TabStop = false;
```

Ustawienia dla nieaktywnego podpisu – dodatkowo ustawiłem wielkość czcionki na *„10pnt”* w celu wygenerowania kodu dla domyślnych ustawień czcionki:

```
213             //
214             // label1
215             //
216             this->label1->BorderStyle = System::Windows::Forms::BorderStyle::FixedSingle;
217             this->label1->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Regular,
218                 System::Drawing::GraphicsUnit::Point, static_cast<System::Byte>(238)));
219             this->label1->Location = System::Drawing::Point(12, 363);
220             this->label1->Name = L"label1";
221             this->label1->Size = System::Drawing::Size(199, 22);
222             this->label1->TabIndex = 3;
223             this->label1->Text = L"cup #0";
224             this->label1->TextAlign = System::Drawing::ContentAlignment::MiddleCenter;
```

Dalej, dla aktywnego kubka:

```
202            //
203            // pictureBox2
204            //
205            this->pictureBox2->BorderStyle = System::Windows::Forms::BorderStyle::Fixed3D;
206            this->pictureBox2->Image = (cli::safe_cast<System::Drawing::Image^>(resources->GetObject(L"pictureBox2.Image")));
207            this->pictureBox2->Location = System::Drawing::Point(236, 41);
208            this->pictureBox2->Name = L"pictureBox2";
209            this->pictureBox2->Size = System::Drawing::Size(203, 323);
210            this->pictureBox2->SizeMode = System::Windows::Forms::PictureBoxSizeMode::AutoSize;
211            this->pictureBox2->TabIndex = 2;
212            this->pictureBox2->TabStop = false;
```

Dla aktywnego podpisu:

```
225            //
226            // label2
227            //
228            this->label2->BorderStyle = System::Windows::Forms::BorderStyle::Fixed3D;
229            this->label2->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold,
230                System::Drawing::GraphicsUnit::Point, static_cast<System::Byte>(238)));
231            this->label2->ForeColor = System::Drawing::Color::FromArgb(static_cast<System::Int32>(static_cast<System::Byte>(192)),
232                static_cast<System::Int32>(static_cast<System::Byte>(0)),
233                static_cast<System::Int32>(static_cast<System::Byte>(0)));
234            this->label2->Location = System::Drawing::Point(240, 363);
235            this->label2->Name = L"label2";
236            this->label2->Size = System::Drawing::Size(199, 22);
237            this->label2->TabIndex = 4;
238            this->label2->Text = L"cup #1";
239            this->label2->TextAlign = System::Drawing::ContentAlignment::MiddleCenter;
```

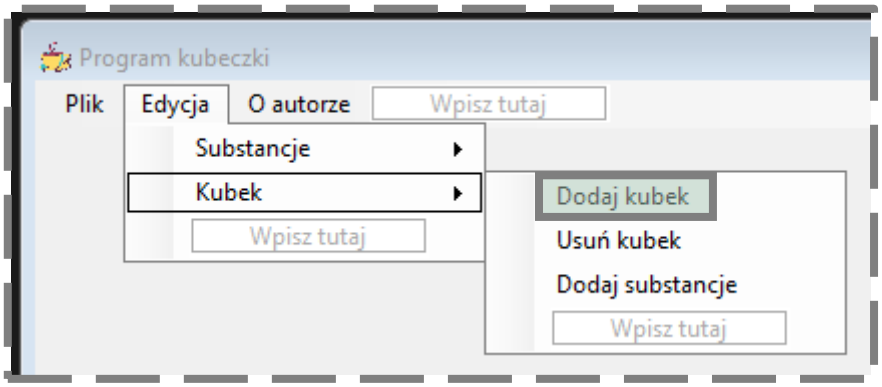
Powyższe działania były konieczne w celu możliwości skopiowania powyższych kodów – zostaną one wykorzystane podczas dynamicznego tworzenia kontrolek.

Przed przystąpieniem do zadań usuń utworzone elementy typu ***PictureBox*** i ***Label***.

2. Zadania

ZADANIE 2.1 – tworzenie kubka: GUI

Kubek będzie tworzony automatycznie po wybraniu opcji w menu aplikacji: *Edycja* → *Kubek* → *Dodaj kubek*:



Kliknij dwukrotnie na element w celu utworzenia szkieletu funkcji w pliku *MainWin.h*:

```
1  private: System::Void dodajKubekToolStripMenuItem_Click(System::Object^
   sender, System::EventArgs^ e) {
2
3  }
```

Wiemy już, że tworzenie nowego kubka wiąże się z utworzeniem dwóch kontrolek: ***PictureBox*** i ***Label***. Stwórzmy odpowiedniki wektorów w C++/CLI dla przechowywania tych elementów. Przejdź na samą górę pliku i wklej w odpowiednie miejsce kod:

```
1  private:
2          Generic::List<PictureBox^>^ cups = gcnew Generic::List<PictureBox^>();
3          Generic::List<Label^>^ lbl_cups = gcnew Generic::List<Label^>();
```

```
4     #pragma once
5
6     namespace POiIOkubeczki {
7
8  >      using namespace ...
14
15 >      /// ...
18        public ref class MainWin : public System::Windows::Forms::Form
19        {
20        private:
21            Generic::List<PictureBox^>^ cups = gcnew Generic::List<PictureBox^>();
22            Generic::List<Label^>^ lbl_cups = gcnew Generic::List<Label^>();
23
```

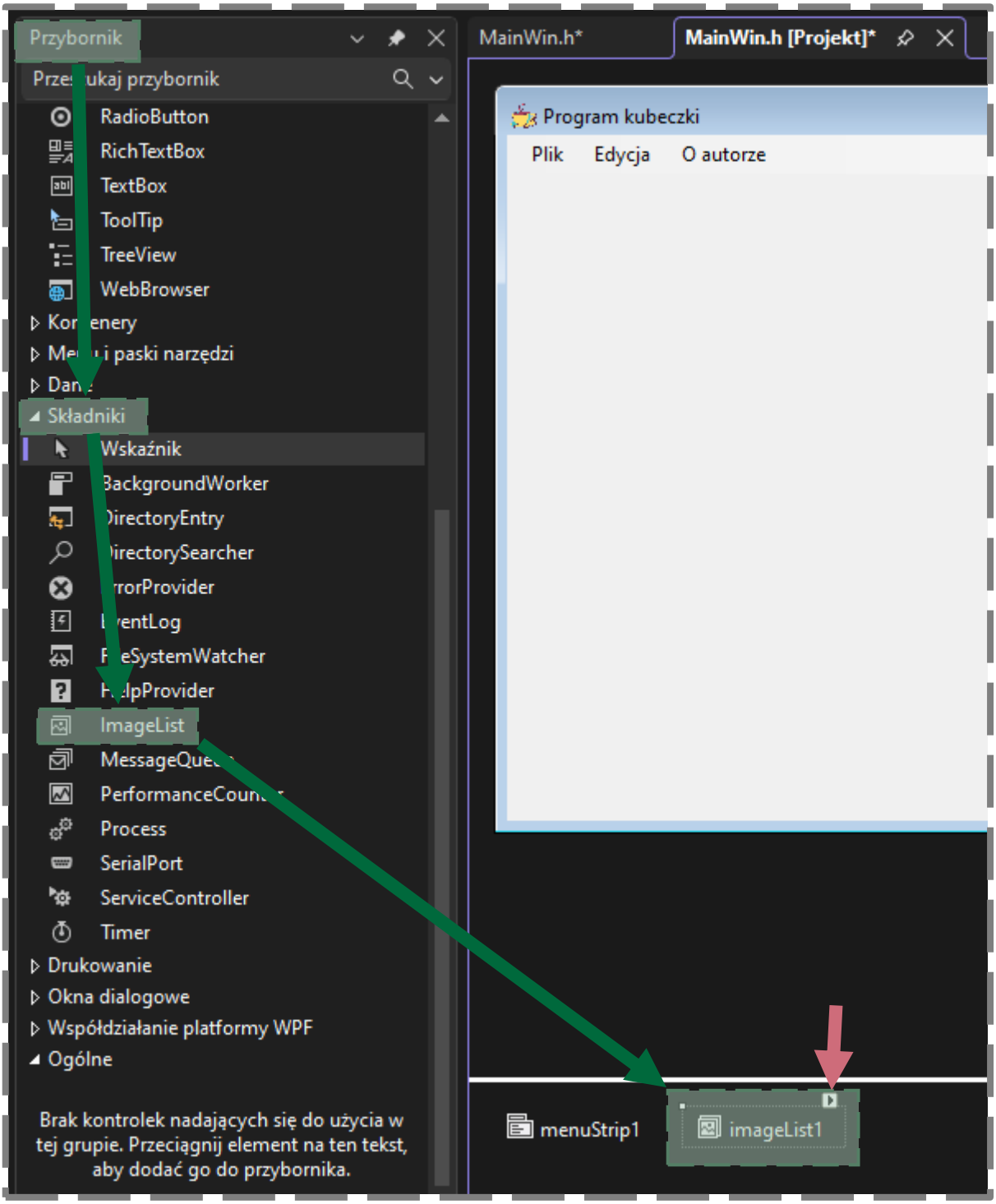
Następnie utwórz funkcję pomocniczą generującą kontrolkę ***PictureBox***:

```
1   private: Void addCup() {
2           PictureBox^ pb = gcnew PictureBox();
3           pb->Size = Drawing::Size(199, 319);
4           pb->SizeMode = System::Windows::Forms::PictureBoxSizeMode::AutoSize;
5           //pb->Image = Image::FromFile("cup.png");
6
7           pb->Location = Point(12, 41);
8           pb->Name = L"cup" + Convert::ToString(cups->Count);
9
10          this->Controls->Add(pb);
11          cups->Add(pb);
12  }
```

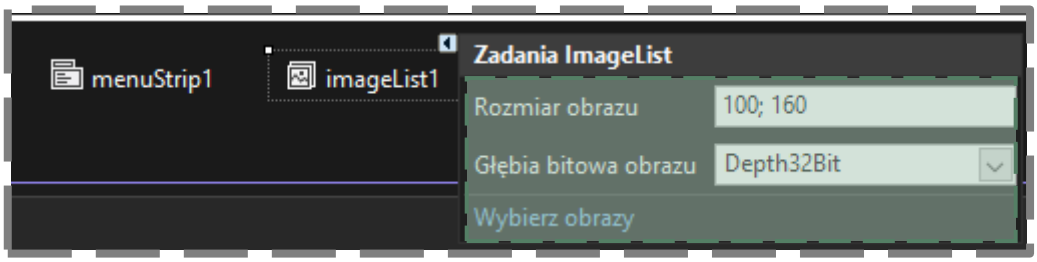
Funkcję można umieścić za poleceniem `#pragma endregion`:

```
65      #pragma region Windows Form Designer generated code
66  >           /// ...
70  >           void InitializeComponent(void) { ... }
198     #pragma endregion
199
200
201     private: Void addCup() {
202         PictureBox^ pb = gcnew PictureBox();
203         pb->Size = Drawing::Size(199, 319);
204         pb->SizeMode = System::Windows::Forms::PictureBoxSizeMode::AutoSize;
205         //pb->Image = Image::FromFile("cup.png");
206
207         pb->Location = Point(12, 41);
208         pb->Name = L"cup" + Convert::ToString(cups->Count);
209
210         this->Controls->Add(pb);
211         cups->Add(pb);
212     }
```

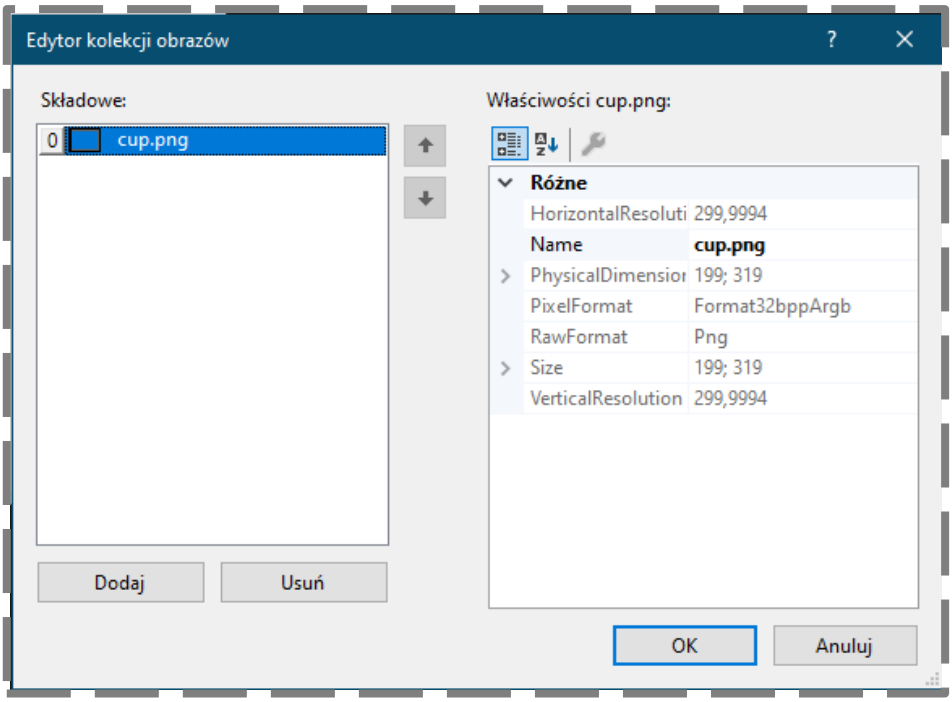
Zwróć uwagę, że linijka 5 w powyższym listningu została zakomentowana. W celu uniknięcia potencjalnych problemów z ścieżką dostępu do pliku, wczytamy go wcześniej i załadujemy do kontrolki ***ImageList*** – kliknij na nią dwukrotnie LPM w oknie ***Przybornika***:



Klikając na strzałkę w górnym prawym rogu kontrolki należy zmienić rozmiar zdjęcia na (199; 200) oraz *Głębię bitową obrazu* na „*Depth32Bit*”.



Następnie kliknij „Wybierz obrazy” i w nowym oknie „Dodaj” – wskaż lokalizację obrazka.



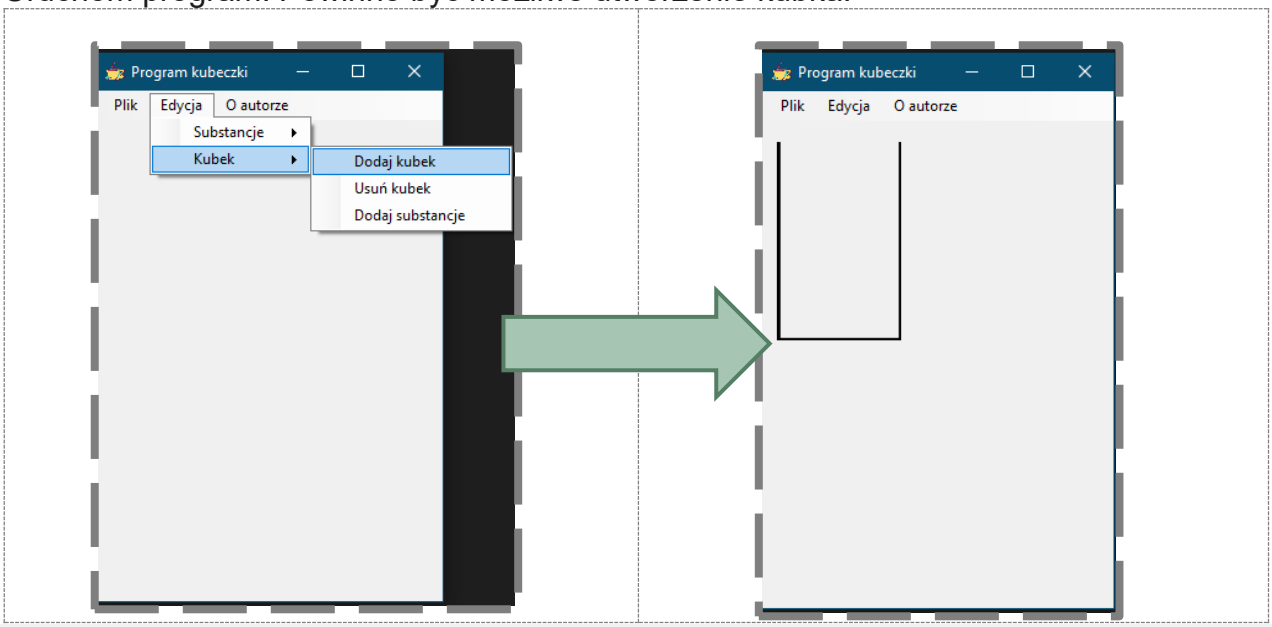
Wróćmy do kodu. W miejsce zakomentowanej linii wstaw:

```
1  pb->Image = imageList1->Images[0];
```

Przejdź do końca pliku, gdzie znajduje się pusta funkcja odpowiedzialna za obsługę zdarzenia kliknięcia na opcję menu „*Dodaj kubek*”. Wstaw do nie wywołanie funkcji *addCup()*:

```
1  private: System::Void dodajKubekToolStripMenuItem_Click(System::Object^
   sender, System::EventArgs^ e) {
2          addCup();
3  }
```

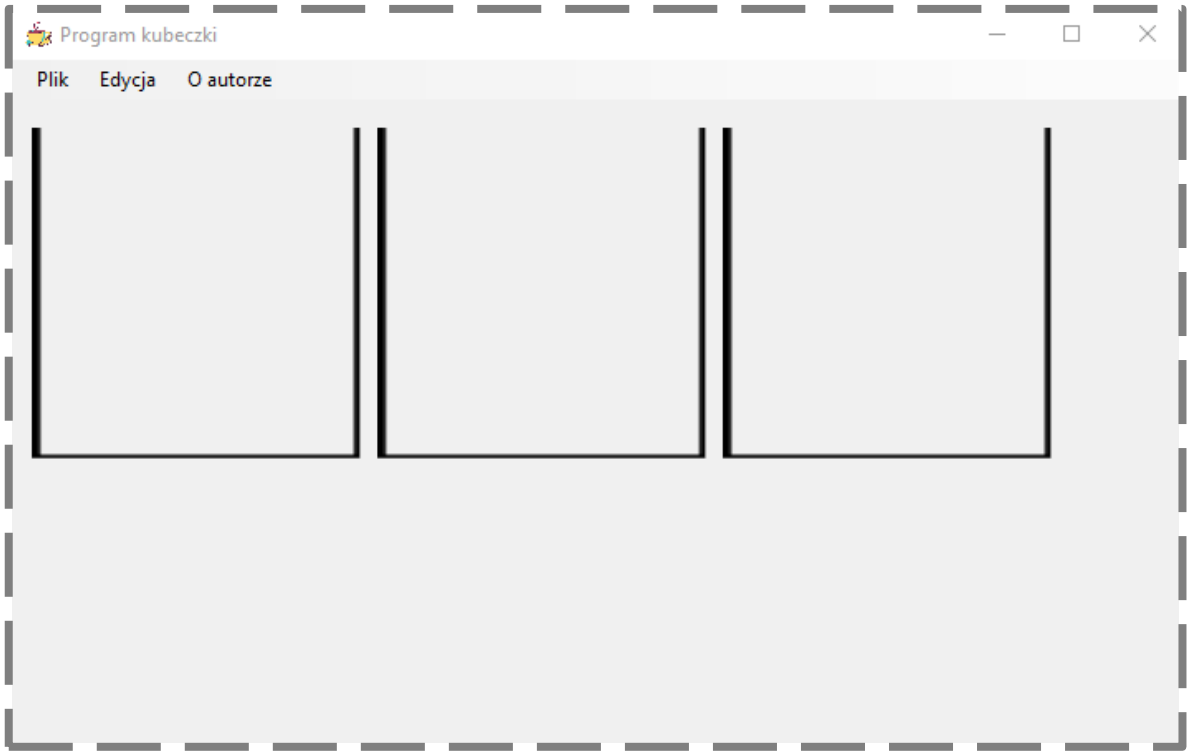
Uruchom program. Powinno być możliwe utworzenie kubka:



Przed utworzeniem nowego kubka musimy jeszcze zmodyfikować pozycję jego wstawienia. Zobacz zmiany w kodzie funkcji *addCup()* – linijka 6. została zastąpiona 7.:

```
1   private: Void addCup() {
            PictureBox^ pb = gcnew PictureBox();
2           pb->Size = Drawing::Size(199, 319);
3           pb->SizeMode = System::Windows::Forms::PictureBoxSizeMode::AutoSize;
4           pb->Image = imageList1->Images[0];
5
6           //pb->Location = Point(12, 41);
7           pb->Location = Point(12 + (10 + 199) * cups->Count, 41);
8           pb->Name = L"cup" + Convert::ToString(cups->Count);
9
10          this->Controls->Add(pb);
11          cups->Add(pb);
12  }
```

Efekt:



Pozostaje dodać funkcję odpowiedzialną za tworzenie kontrolek typu ***Label***. Wstaw kod poniżej funkcji *addCup()*:

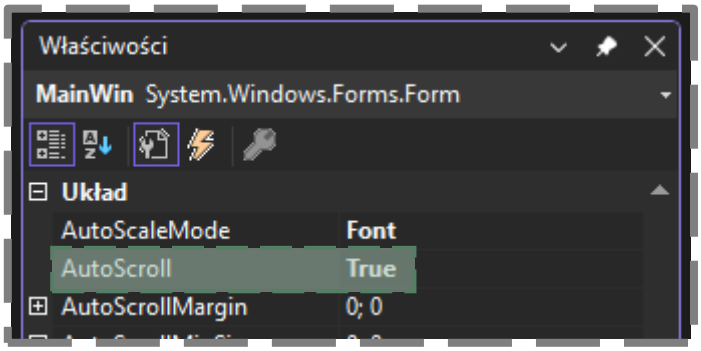
```
1   private: Void addLblCup() {
            Label^ lbl = (gcnew System::Windows::Forms::Label());
2           lbl->BorderStyle = System::Windows::Forms::BorderStyle::FixedSingle;
3           lbl->Size = System::Drawing::Size(199, 22);
4           lbl->TextAlign = System::Drawing::ContentAlignment::MiddleCenter;
5
6
7           lbl->Location = System::Drawing::Point(12 + (10 + 199) * lbl_cups-
    >Count, 243);
8           lbl->Name = L"lblCup" + Convert::ToString(lbl_cups->Count);
9           lbl->Text = L"cup #" + Convert::ToString(lbl_cups->Count);
10
11          this->Controls->Add(lbl);
12          lbl_cups->Add(lbl);
13  }
```

Dodaj wywołanie funkcji *addLblCup()* po kliknięciu opcji menu „*Dodaj kubek*”:

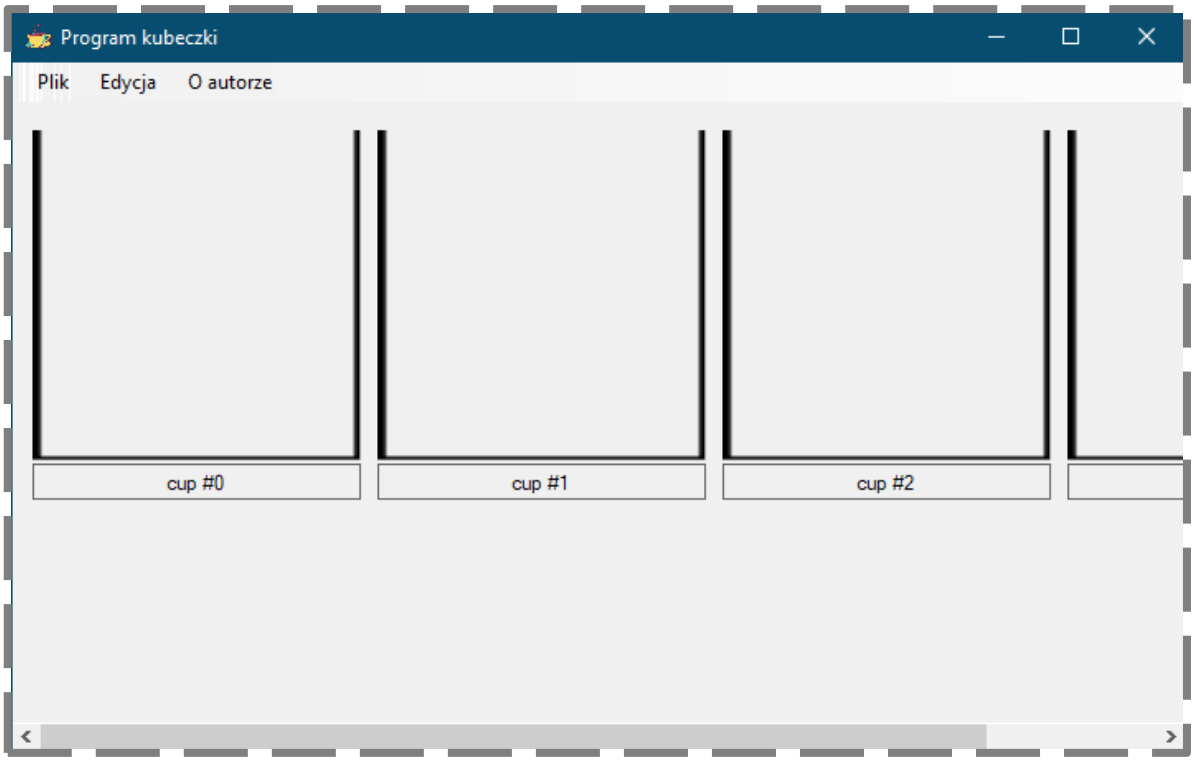
```
1  private: System::Void dodajKubekToolStripMenuItem_Click(System::Object^
   sender, System::EventArgs^ e) {
2          addCup();
3          addLblCup();
4  }
```

Efekt:

Ostatnim problemem do rozwiązania w tym zadaniu jest kwestia znikających kubeczków poza obszar okna aplikacji. W oknie Właściwości dla formy (okna aplikacji) zmień opcję *AutoScroll* na „*True*”:



Efekt końcowy zadania:



Niezależnie od wielkości okna aplikacji, mamy możliwość przejrzenia całej jego zawartości przemieszczając się suwakami.

Prześlij zmiany na repozytorium zdalne.

ZADANIE 2.2 – wybór kubka

Zajmiemy się teraz okodowaniem akcji wyboru kubka. Tylko dla aktywnego, wybranego kubka będzie możliwość dodania substancji. Zacznijmy od napisania funkcji obsługującej zdarzenia pojedynczego kliknięcia LPM na etykietę kubka – skopiuj szkielet ostatniej funkcji w pliku *MainWin.h* i zmień jej nazwę na *selectCup*:

```
255     private: System::Void dodajKubekToolStripMenuItem_Click(System::Object^ sender, System::EventArgs^ e) {
256         addCup();
257         addLblCup();
258     }
259
260     private: System::Void selectCup(System::Object^ sender, System::EventArgs^ e) {
261
262     }
263
264     };
265     }
```

Wstaw kod funkcji:

```
1  private: System::Void selectCup(System::Object^ sender, System::EventArgs^ e)
2  {
3          Label^ lbl = (Label^)sender;
4          lbl->BorderStyle = System::Windows::Forms::BorderStyle::Fixed3D;
5          lbl->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 12,
   System::Drawing::FontStyle::Bold, System::Drawing::GraphicsUnit::Point,
   static_cast<System::Byte>(238)));
6          lbl->ForeColor = System::Drawing::Color::FromArgb(255, 0, 0);
7  }
```

W celu możliwości zmiany parametrów etykiety w funkcji, należy wskazać, że kliknięty obiekt `sender` jest typu ***Label***:

```
Label^ lbl = (Label^)sender;
```

Pamiętaj, że tworzenie zmiennych złożonego typu (elementy GUI, String itd.) wymagają w C++/CLI uchwytu do elementu – znaku „^” po określonym typie zmiennej:

```
Label^ lbl
```

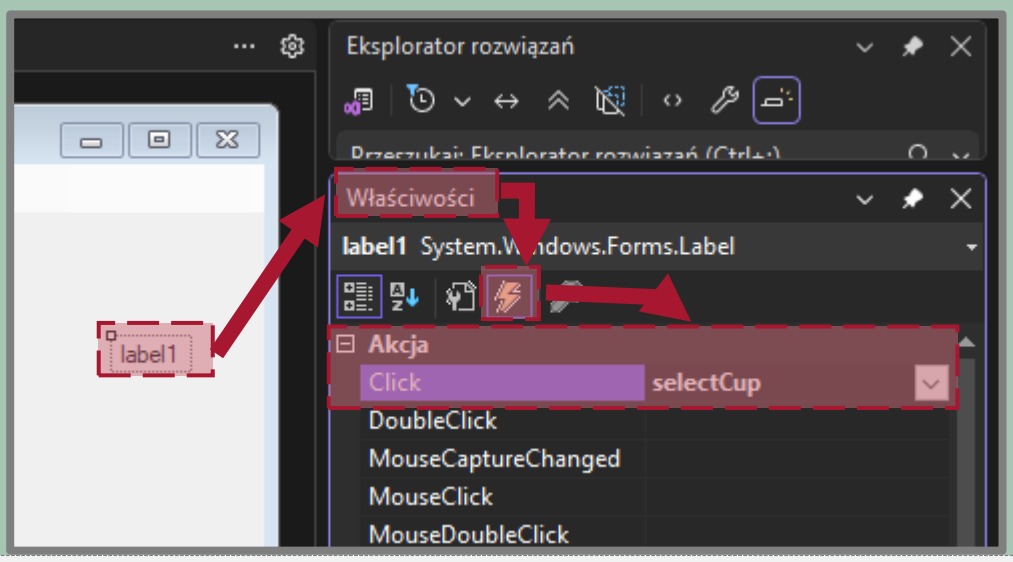
Dalej, za pomocą nazwy zmiennej i strzałki „->” można odwołać się do dowolnej opcji kontrolki określonej w oknie ***Właściwości***, np.:

```
lbl->ForeColor = System::Drawing::Color::FromArgb(255, 0, 0);
```

Pozostaje wskazać funkcję *selectCup* jako funkcję obsługującą zdarzenie kliknięcia LPM na etykietę. W funkcji tworzenia etykiety dodaję odpowiednią instrukcję:

```
1  private: Void addLblCup() {
             Label^ lbl = (gcnew System::Windows::Forms::Label());
2            lbl->BorderStyle = System::Windows::Forms::BorderStyle::FixedSingle;
3            lbl->Size = System::Drawing::Size(199, 22);
4            lbl->TextAlign = System::Drawing::ContentAlignment::MiddleCenter;
5
6
7            lbl->Location = System::Drawing::Point(12 + (10 + 199) * lbl_cups-
   >Count, 243);
8            lbl->Name = L"lblCup" + Convert::ToString(lbl_cups->Count);
9            lbl->Text = L"cup #" + Convert::ToString(lbl_cups->Count);
10
11           lbl->Click += gcnew System::EventHandler(this, &MainWin::selectCup);
12
13           this->Controls->Add(lbl);
14           lbl_cups->Add(lbl);
15 }
```

Zapomniałem sprawdzić wcześniej sposobu przypisania funkcji do obsługi zdarzenia. W tym celu chwilowo tworzę etykietę w oknie aplikacji. W oknie ***Właściwości*** etykiety przechodzę do zakładki ***zdarzeń*** i dla akcji *Click* wybieram naszą funkcję:



W kodzie pliku *MainWin.h* odnajduję wywołanie zdarzenia:

```
194            //
195            // label1
196            //
197            this->label1->AutoSize = true;
198            this->label1->Location = System::Drawing::Point(636, 88);
199            this->label1->Name = L"label1";
200            this->label1->Size = System::Drawing::Size(35, 13);
201            this->label1->TabIndex = 1;
202            this->label1->Text = L"label1";
203            this->label1->Click += gcnew System::EventHandler(this, &MainWin::selectCup);
```

Lewą stronę instrukcji skopiowałem powyżej do funkcji tworzącej nową etykietę.

Po sprawdzeniu jak dodać akcję do kontrolki, usuń utworzoną w ramach testów etykietę. Kod z nią powiązany zostanie usunięty automatycznie.

Uruchom program:

Jak widać brakuje opcji *odkliknięcia* etykiety. W tym celu utworzymy nową funkcję, poniżej funkcji *addLblCup*:

```
1  private: Void cleanLblCup() {
2  for each (Label ^ lbl in lbl_cups)
3          {
4          lbl->BorderStyle = System::Windows::Forms::BorderStyle::FixedSingle;
5          lbl->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 8,
   System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
   static_cast<System::Byte>(238)));
6          lbl->ForeColor = System::Drawing::Color::FromArgb(0, 0, 0);
7          }
8  cupID = -1;
9  }
```

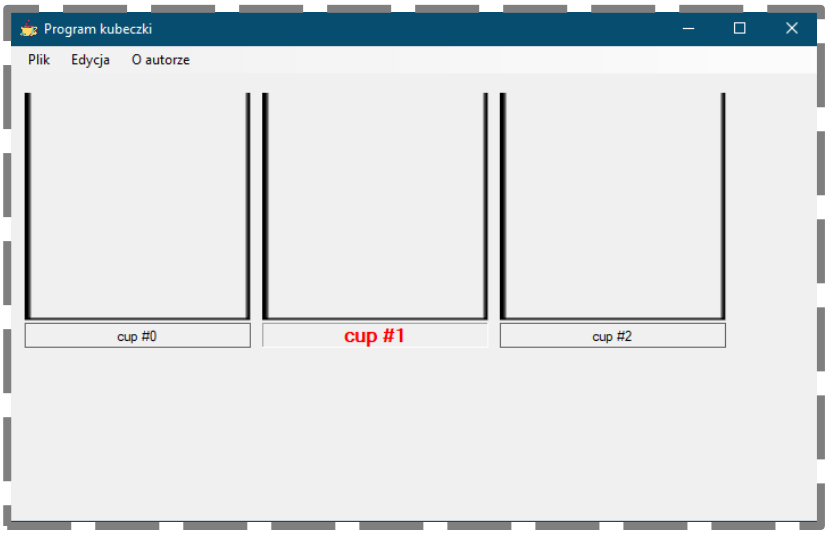
Zwróć uwagę, że w 8. linijce powyższego listningu została użyta zmienna pomocnicza *cupID*. Musimy ją zdefiniować na początku pliku:

```
22        public ref class MainWin : public System::Windows::Forms::Form
23        {
24        private:
25            Generic::List<PictureBox^>^ cups = gcnew Generic::List<PictureBox^>();
26            Generic::List<Label^>^ lbl_cups = gcnew Generic::List<Label^>();
27            int cupID = -1;
```

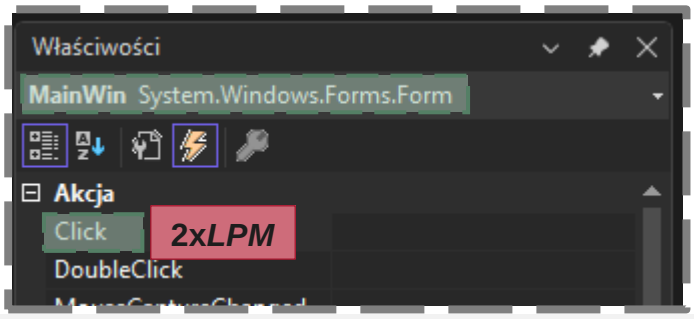
Pozostaje wywołać funkcję *cleanLblCup* w funkcji obsługi zdarzenia *selectCup*:

```
1  private: System::Void selectCup(System::Object^ sender, System::EventArgs^ e)
2  {
3          cleanLblCup();
4
5          Label^ lbl = (Label^)sender;
6          lbl->BorderStyle = System::Windows::Forms::BorderStyle::Fixed3D;
7          lbl->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 12,
   System::Drawing::FontStyle::Bold,System::Drawing::GraphicsUnit::Point,
   static_cast<System::Byte>(238)));
8          lbl->ForeColor = System::Drawing::Color::FromArgb(255, 0, 0);
9  }
```

Uruchom aplikację i przetestuj – tym razem istnieje możliwość wyróżnienia tylko jednej etykiety:



Brakuje możliwości odznaczenia wszystkich kubków poprzez kliknięcie w pustą przestrzeń okna aplikacji. W tym celu kliknij na okno, przejdź w ***Właściwościach*** do akcji *Click* i dwukrotnie przeciśnij LPM – zostanie utworzona funkcja obsługująca zdarzenie. Wstaw do niej wywołanie funkcji *cleanLblCup*:



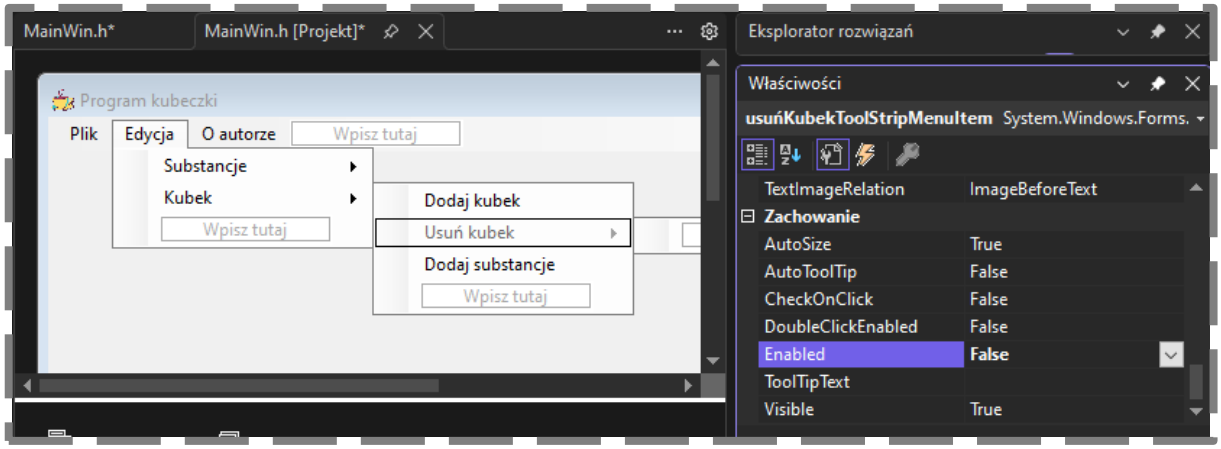
```
MainWin.h*   MainWin.h [Projekt]*
POiIO kubeczki                    (Globalny zasięg)
284
285    private: System::Void MainWin_Click(System::Object^ sender, System::EventArgs^ e) {
286        cleanLblCup();
287    }
```

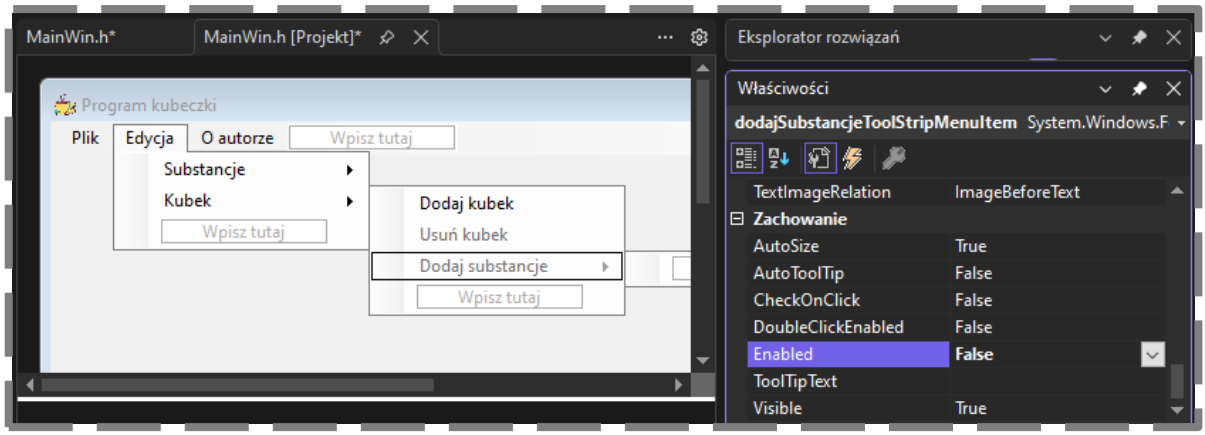
Uruchom aplikację i sprawdź jej działanie.

Prześlij zmiany na repozytorium zdalne.

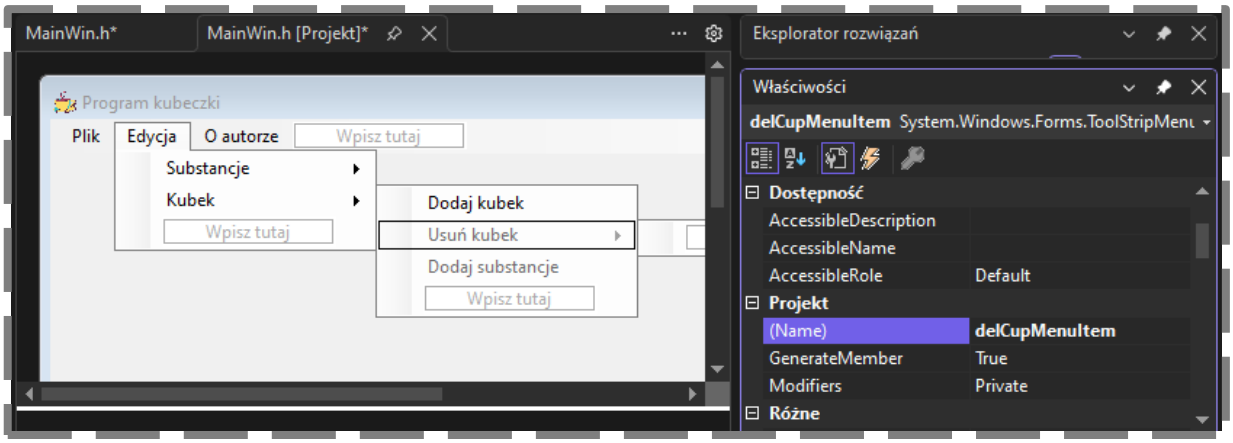
ZADANIE 2.3 – dynamiczna zmiana właściwości opcji menu

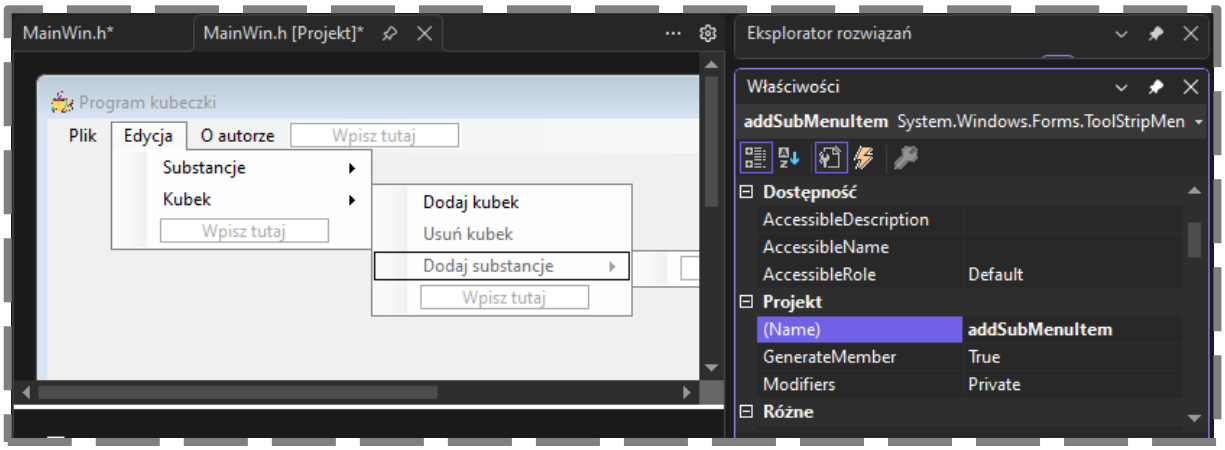
Możliwość usuwania kubka lub dodawania do niego zawartości powinna być dostępna jedynie w sytuacji, gdy któryś z utworzonych kubków jest aktywny. Domyślnie więc opcje te powinny nieaktywne. Dla obu wspomnianych przycisków, w oknie ***Właściwości***, ustaw *Enabled* na „*False*”:





Zmień nazwy opcji menu w oknie ***Właściwości***, aby łatwiej było się do nich odwołać:





Do kodu funkcji *selectCup* dodaj instrukcje:

```
private: System::Void selectCup(System::Object^ sender, System::EventArgs^ e)
{
        cleanLblCup();

        Label^ lbl = (Label^)sender;
        lbl->BorderStyle = System::Windows::Forms::BorderStyle::Fixed3D;
        lbl->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold, System::Drawing::GraphicsUnit::Point, static_cast<System::Byte>(238)));
        lbl->ForeColor = System::Drawing::Color::FromArgb(255, 0, 0);

        getCupID(lbl);
        delCupMenuItem->Enabled = true;
        delCupMenuItem->Text = L"Usuń kubek #" + Convert::ToString(cupID);
        addSubMenuItem->Enabled = true;
        addSubMenuItem->Text = L"Dodaj substancje do kubka #" + Convert::ToString(cupID);
}

```

Zwróć uwagę, że utworzyłem nową funkcję *getCupID(lbl)* – linijka 10. Jej kod należy wstawić tuż za funkcją *cleanLblCup()*:

```
1  private: Void getCupID(Label^ lbl) {
2          int count = lbl_cups->Count;
3          int _id = -1;
4          bool run = true;
5
6          while (run)
7          {
8                  _id += 1;
9
10                 if (lbl == lbl_cups[_id]) run = false;
11                 else if ((_id + 1 == count) && (run == true))
12                 {
13                         _id = -1;
14                         run = false;
15                 }
16         }
17
18         cupID = _id;
19 }
20
```

Za jej pomocą przechwytuję informację o pozycji etykiety (i obrazu kubka) w liście `lbl_cups`. Pozycja na liście zapisywana jest do zmiennej `cupID`, której definicja została dodana u góry pliku:

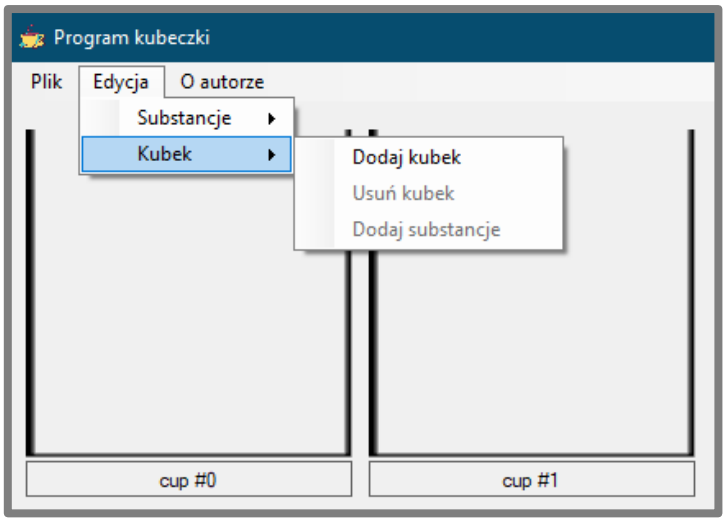
```
6   namespace POiIOkubeczki {
7
8   >   using namespace ...
14
15  >   /// ...
18      public ref class MainWin : public System::Windows::Forms::Form
19      {
20      private:
21          Generic::List<PictureBox^>^ cups = gcnew Generic::List<PictureBox^>();
22          Generic::List<Label^>^ lbl_cups = gcnew Generic::List<Label^>();
23          int cupID = -1;
24
25      private: System::Windows::Forms::ImageList^ imageList1;
26
```

Należy jeszcze deaktywować opcje menu i zmienić ich tekst w przypadku, gdy żaden kubek nie jest aktywny:

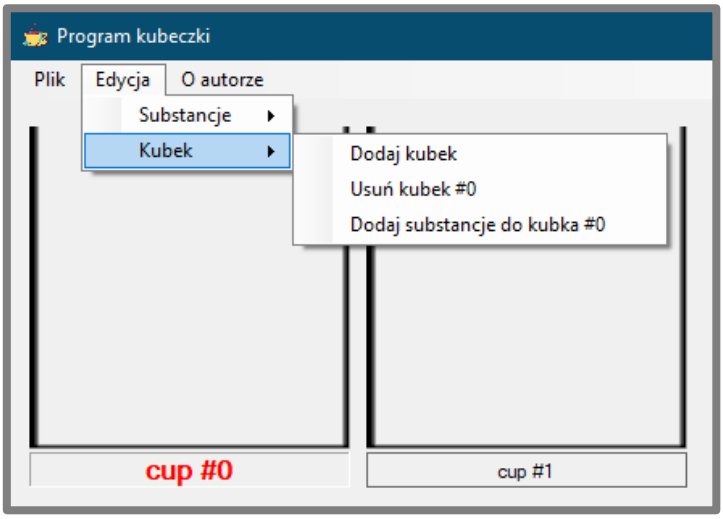
```
1  private: Void cleanLblCup() {
2  for each (Label ^ lbl in lbl_cups)
3  {
4          lbl->BorderStyle = System::Windows::Forms::BorderStyle::FixedSingle;
5          lbl->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 8,
   System::Drawing::FontStyle::Regular,
6          System::Drawing::GraphicsUnit::Point,
   static_cast<System::Byte>(238)));
7          lbl->ForeColor = System::Drawing::Color::FromArgb(0, 0, 0);
8  }
9
10         delCupMenuItem->Enabled = false;
11         delCupMenuItem->Text = L"Usuń kubek";
12         addSubMenuItem->Enabled = false;
13         addSubMenuItem->Text = L"Dodaj substancje";
14         cupID = -1;
15 }
16
```

Uruchom aplikację i sprawdź czy wszystko działa poprawnie:

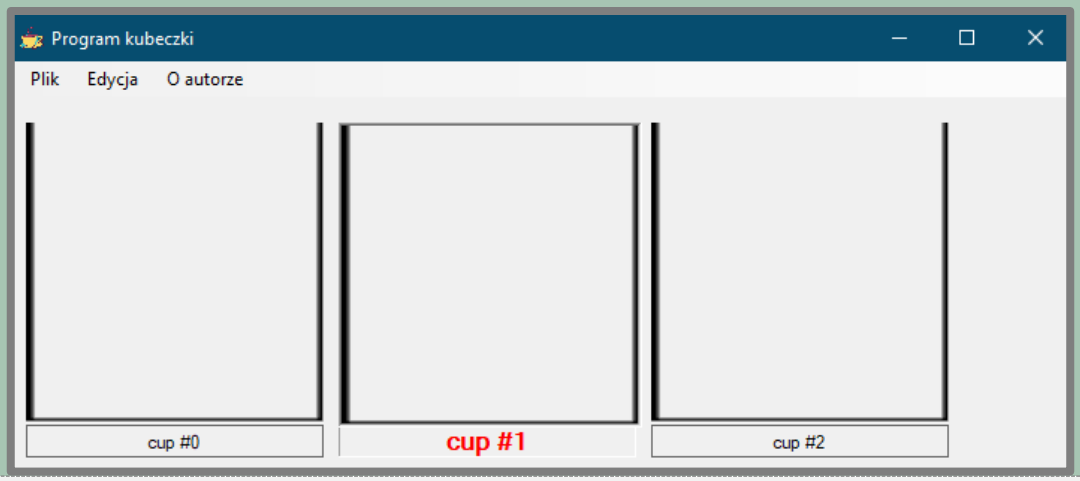
- dla nieaktywnego kubka



- dla aktywnego kubka

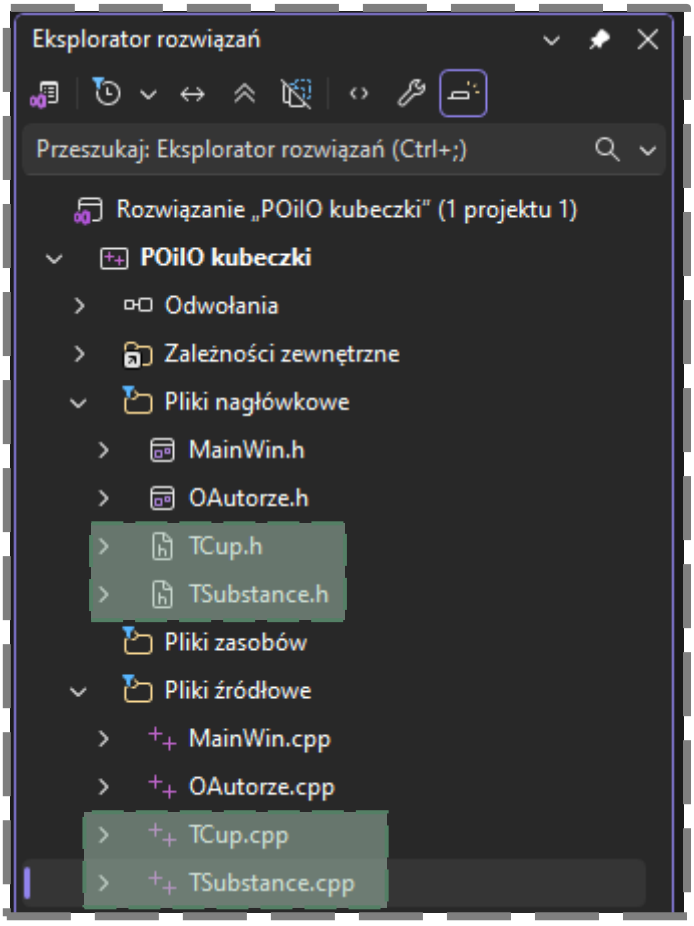


Możesz spróbować samodzielnie dodać kod odpowiedzialny za dodanie ramki do obrazka aktywnego kubka:



ZADANIE 2.4 – tworzenie kubka: warstwa logiczna, backend

Do projektu dodaj dwie puste klasy: *TCup* oraz *TSubstance*:



Wstaw do nich kod ze swojego poprzedniego projektu. W plikach klasy *TCup* zdefiniuj wektor wskaźników do obiektów klasy *TCup*:

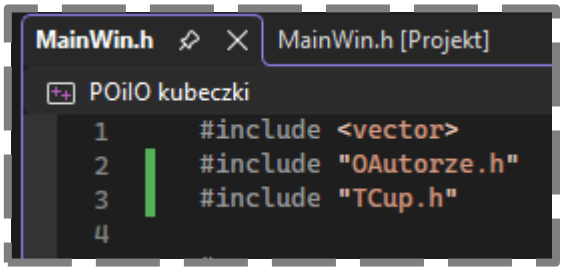
- na końcu pliku nagłówkowego *TCup.h*:

```
extern std::vector<TCup*> cups_pnt;
```

- na końcu / początku pliku źródłowego *TCup.cpp*:

```
vector<TCup*> cups_pnt;
```

Na początku pliku *MainWin.h* dołącz plik nagłówkowy *TCup.h*:



Powyżej funkcji *addCup()* (w pliku *MainWin.h*) zdefiniuj funkcję *addTCup()*:

```
private: Void addTCup() {
        TCup cup;
        cups_pnt.push_back(&cup);
}

```

Dodaj wywołanie nowej funkcji *addTCup()* w funkcji obsługującej akcję wciśnięcia opcji menu „*Dodaj kubek*”:

```
private: System::Void dodajKubekToolStripMenuItem_Click(System::Object^
sender, System::EventArgs^ e) {
        addTCup();
        addCup();
        addLblCup();
}

```

Uruchom program i sprawdź poprawność jego działania. Pomimo braku dodatkowych, widocznych akcji w aplikacji udało się dołączyć warstwę logiczną dla obsługi kubków. W kolejnej instrukcji zobaczysz jak dodać substancje do kubków.

3. Podsumowanie

Tematem niniejszej instrukcji była obsługa zdarzenia skojarzonego z przyciśnięciem opcji menu „Dodaj kubek”. Nauczyłeś się dynamicznie dodawać elementy GUI w trakcie działania programu i odpowiednio je rozlokować w oknie aplikacji. Dużo czasu poświęciliśmy na obsługę kliknięcia wybranego kubka – aktywowanie/deaktywowanie wyboru, odpowiednio przy tym modyfikując opcję menu.

Jedną z istotniejszych kwestii poruszanych w tej instrukcji było współtworzenie kodu z projektantem GUI środowiska Visual Studio. Umiejętność wyszukiwania pożądanych właściwości elementów GUI, edycji automatycznie wygenerowanego kodu i nazw kontrolek znacznie przyśpieszają pracę nad aplikacją i pozwalają zrozumieć ideę pracy z językiem C++/CLI.